

Visualization

The visualization comprises the visualization of the instrument and the visualization of trajectories.

General

In principle, there are 4 different ways to run the simulation: normal run, instrument visualization, visualization of trajectories and a test run – Table 1 summarizes the main features of these options.

	bVisInstr	bVisTraj	bTest	instrument.inf	features
normal run	FALSE	FALSE	FALSE	written	
instrument visualization	TRUE	FALSE	FALSE	written	
trajectory visualization	FALSE	TRUE	FALSE	needed	
check/test	FALSE	FALSE	TRUE	written	no treatment of trajectories

Table 1: Main features of the 4 options to run a VITESS simulation

In the test run no trajectories are treated, but all checks are executed, all input files for the simulation are written and *instrument.inf* is generated. It can be used to start the trajectory visualization, because this needs *instrument.inf* to run. Writing this file only works at the end of the simulation, but the visualization of trajectories needs it during the simulation.

For the moment, instrument visualization is performed during a normal run, so that there are only 3 options realized.

The following global parameters record the position and orientation of the component in the absolute co-ordinate system:

```
double   Length,           /* length of beamline from source to origin of this module [cm] */
          RotZ, RotY;       /* hor. and vert. rotation of the local co-ordinate system relative
to the absolute one */
double   RotMatrixM[3][3], /* matrix to rotate between these co-ordinate systems */
          RotMatrixS[3][3]; /* matrix to rotate from abs. co-ordinate system to co-ordinate
system of last section */
VectorType BegPosM,        /* end position of prev. module = origin of this module in absolute
co-ordinate system [cm] */
          BegPosS;         /* end position of prev. section = origin of this section in
absolute co-ordinate system [cm] */
```

BegPosS and *RotMatrixS* are only used in Guide, because this is the only module that treats elements in local co-ordinate systems.

The unit for positions in the output files is meter.

Test or Check run

Writing the intersection points requires that *instrument.inf* already exists up to the preceding module, i.e. that a calculation of the whole instrument had been executed before, but of course a simulation of trajectories is not necessary. This is controlled by the parameter *bTest*. It is declared in *init.c* and set to *FALSE* by default. If the parameter *--t* is set, it is changed to *TRUE* in *Init()*. *bTest* prevents reading of trajectories in *ReadNeutrons()*. In this way, only checks are executed and the input and inf files are generated.

This also allows the user to check the generated input files, e.g. the guide shape resulting from the input for an elliptic or parabolic guide, before he starts the simulation.

Visualization of the Instrument.

The instrument visualization is controlled by the parameter *bVisInstr*. It is declared in *init.c* and set to *FALSE* by default. For the time being, the default is *TRUE* and the default name is *geometry.inf*. If the parameter *--vinstr-file* is set, *bVisInstr* is *TRUE* and the file name is set to *instr-file* in *Init()*.

The structure that has to be filled is *VtModGeom stGeometry*. *VtModGeom* and all structures that it contains are defined in *general.h* as. *stGeometry* is declared in *init.c* as a global variable. *VtModGeom* contains pointers to all geometrical elements that can be visualized and their numbers:

```
typedef struct
{
    VtModID      eModule;
    VtLine*      pLine;
    int          nLines;
    VtRectangle* pRectangle;
    int          nRectangles;
    VtOpenRect*  pOpenRect;
    int          nOpenRects;
    VtCircle*    pCircle;
    int          nCircles;
    VtCuboid*    pCuboid;
    int          nCuboids;
    VtHull*      pHull;
    int          nHulls;
    VtCylinder*  pCylinder;
    int          nCylinders;
    VtHolCyl*    pHolCyl;
    int          nHolCyls;
    VtEllipsoid* pEllipsoid;
    int          nEllipsoids;
    VtSphere*    pSphere;
    int          nSpheres;
    const char*  pDescr; /* description */
}
VtModGeom;
```

VtHolCyl is a hollow cylinder. *VtHull* has rectangular top and bottom (or entrance and exit), with the normal through the centers coinciding and the edges parallel; the corners of the rectangles are connected by straight lines. For details of any geometrical element see structure definitions in *general.h*.

Each module has to fill this structure at the end of the program, but before *Cleanup()* is called, e.g. in *OwnCleanup()*. Here all positions and orientations of the elements describing the component are given in a relative coordinate system.

The file *instr-file* is then opened and one line appended in the function *WriteGeomData()* after writing the instrument file *instrument.inf* in *WriteInstrData()*, both called in the function *Cleanup()*. *WriteGeomData()* takes the absolute position of the end of the previous module

from *instrument.inf*, which is read in *ReadInstrData()*, and calculates the absolute values of the elements from these positions and the relative positions given in *stGeometry*.

Therefore, writing the geometry file works in the same way as writing *instrument.inf* and should therefore not be a problem.

The output files look like this:

```
#
#units
# [m] position, length, width, height, radius
# [deg] angels
#
Circle      0.00000 -0.11000  0.00000  1.00000  0.00000  0.00000  0.07750  0.00000  360.00000  Source
Circle      0.00000 -0.11000  0.00000  1.00000  0.00000  0.00000  0.07750  0.00000  360.00000  Source
Rectangle   0.00000 -0.04500  0.00000  1.00000  0.00000  0.00000  0.34600  0.20700  Source
Rectangle   1.00000  0.00000  0.00000  1.00000  0.00000  0.00000  0.06000  0.10000  Source
Hull        1.49997  0.04500  0.00000  1.00000  0.00000  0.00000  1.00000  0.06000  0.06000  0.10000  0.10000  guide
Hull        2.49992  0.04600  0.00000  1.00000  0.00200  0.00000  1.00000  0.06000  0.06000  0.10000  0.10000  guide
Hull        3.49986  0.04900  0.00000  0.99999  0.00400  0.00000  1.00000  0.06000  0.06000  0.10000  0.10000  guide
Hull        4.49980  0.05400  0.00000  0.99998  0.00600  0.00000  1.00000  0.06000  0.06000  0.10000  0.10000  guide
Hull        5.49975  0.06100  0.00000  0.99997  0.00800  0.00000  1.00000  0.06000  0.06000  0.10000  0.10000  guide
Hull        6.49969  0.07000  0.00000  0.99995  0.01000  0.00000  1.00000  0.06000  0.06000  0.10000  0.10000  guide
Hull        7.49963  0.08100  0.00000  0.99993  0.01200  0.00000  1.00000  0.06000  0.06000  0.10000  0.10000  guide
Hull        8.49958  0.09400  0.00000  0.99990  0.01400  0.00000  1.00000  0.06000  0.06000  0.10000  0.10000  guide
Hull        9.49952  0.10900  0.00000  0.99987  0.01600  0.00000  1.00000  0.06000  0.06000  0.10000  0.10000  guide
Hull       10.49946  0.12600  0.00000  0.99984  0.01800  0.00000  1.00000  0.06000  0.06000  0.10000  0.10000  guide
Cylinder    11.24996  0.13950  0.00000  0.99984  0.01799  0.00000  0.50000  0.05000  space
Hull       11.99992  0.15300  0.00000  0.99984  0.01799  0.00000  1.00000  0.06000  0.05831  0.10000  0.10853  guide
Hull       12.99976  0.17099  0.00000  0.99984  0.01799  0.00000  1.00000  0.05831  0.05657  0.10853  0.11411  guide
Hull       13.99960  0.18898  0.00000  0.99984  0.01799  0.00000  1.00000  0.05657  0.05477  0.11411  0.11715  guide
Hull       14.99943  0.20698  0.00000  0.99984  0.01799  0.00000  1.00000  0.05477  0.05292  0.11715  0.11786  guide
Hull       15.99927  0.22497  0.00000  0.99984  0.01799  0.00000  1.00000  0.05292  0.05099  0.11786  0.11627  guide
Hull       16.99911  0.24296  0.00000  0.99984  0.01799  0.00000  1.00000  0.05099  0.04899  0.11627  0.11229  guide
Hull       17.99895  0.26096  0.00000  0.99984  0.01799  0.00000  1.00000  0.04899  0.04690  0.11229  0.10566  guide
Hull       18.99879  0.27895  0.00000  0.99984  0.01799  0.00000  1.00000  0.04690  0.04472  0.10566  0.09581  guide
Hull       19.99862  0.29694  0.00000  0.99984  0.01799  0.00000  1.00000  0.04472  0.04243  0.09581  0.08160  guide
Hull       20.99846  0.31494  0.00000  0.99984  0.01799  0.00000  1.00000  0.04243  0.04000  0.08160  0.06000  guide
Circle     21.69797  0.32760 -0.25000  0.99984  0.01799  0.00000  0.35000  0.00000  360.00000  disc chopper
Circle     21.69797  0.32760 -0.25000  0.99984  0.01799  0.00000  0.20000  85.00000  95.00000  disc chopper
Line       21.69438  0.52681 -0.23257  21.69169  0.67621 -0.21950  disc chopper
Line       21.69438  0.52681 -0.26743  21.69169  0.67621 -0.28050  disc chopper
Rectangle  22.49787  0.34239  0.00000  0.99984  0.01799  0.00000  0.02000  0.03000  slit
Rectangle  22.49800  0.34200  0.00000  0.99984  0.01799  0.00000  0.15000  0.15000  space
```

The parameter *bVisInstalled* handles if the visualization option is included in a module or not. The initial value is *FALSE*; therefore all modules that have visualization implemented have to set:

```
bVisInstalled = TRUE;
```

For the case *bVisInstalled* = *FALSE*, a square perpendicular to the beamline is drawn for modules of length zero, and a cylinder with the beamline axis as symmetry axis is drawn for modules with length greater zero. (This means that for some modules the implementation can be omitted.)

Visualization of Trajectories

The visualization of the trajectories is controlled by the parameter *bVisTraj*. It is declared in *init.c* and set to *FALSE* by default. If the parameter *--Vtraj-file* is set, it is changed to *TRUE* in *Init()*. At the same time the trajectory file is opened and the file pointer *TrajFilePtr* filled.

Writing of interaction points into the trajectory file *traj-file* is carried out in the function *WriteWWP(Neutron *pNeutron, VtReason eReason)* in *init.c*. *VtReason* is defined in *general.h* and can 8 different values (*VT_CREATED*, *VT_OUTSIDE*, *VT_OUT_OF_WND*, *VT_PASSED*, *VT_ENTERED*, *VT_TRANSIT*, *VT_REFLECTED*, *VT_SCATTERED*, *VT_ABSORBED*, *VT_EXITED*); it expresses what has happened with the neutron at the intersection point.

In order not to slow down the simulation, it has been decided to use the parameter *bVisTraj* in the calling program, i.e. only to call *WriteWWP()*, if trajectories are requested:

```
if (bVisTraj) WriteWWP(pNeutron, eReason);
```

This can be written shorter using a macro (defined in *init.h*):

```
WriteIAP(pNeutron, eReason);
```

WriteWWP()

- writes a header in the first call
- counts the number of trajectories and limits them to *MAX_TRAJ*
- calculates the neutron position in the absolute co-ordinate system
- writes ID, color, wavelength, intensity loss, position, spin-state and reason into the file

The intensity loss is supposed to be used later on by another tool to estimate the radiation background.

WriteWWP() has to be called at each point of entering, exiting, reflection, scattering or absorption, and also if the entrance area is not hit. *VT_PASSED* is supposed to be used if entrance and exit point are identical (disc chopper, slit, etc.), *VT_TRANSIT* for the transition from one segment to the next (guide).

It is essential that the definition of the new co-ordinate system (*Position[0]=0.0*) is executed **after** writing all interaction points.

The output file looks like this:

#	ID	color	lambda	pos x	pos y	pos z	spin	reason
#	-----	-----	-----	-----	-----	-----	-----	-----
	AA0000000001	0	5.74683	350.000	-1.173	2.146	-1	2
	AA0000000002	0	4.14582	350.000	3.894	-0.990	-1	0
	AA0000000003	0	3.54095	350.000	-7.399	-4.174	-1	0
	AA0000000004	0	6.90066	350.000	-7.807	-0.058	1	0
	AA0000000005	0	2.64564	350.000	-5.606	1.265	1	0
	AA0000000006	0	7.83605	350.000	4.681	-6.721	1	0
	AA0000000007	0	1.35295	350.000	3.168	8.873	1	0
	AA0000000008	0	2.95180	350.000	-14.700	12.990	1	0
	AA0000000009	0	3.17460	350.000	-10.202	11.749	-1	0
	AA0000000010	0	7.05914	350.000	10.942	-2.399	1	0

The *traj-file* is closed again in *Cleanup()*. So far this has been realized for the modules *Source*, *Slit*, *Chopper_disc* and *Guide*.